Git & GitHub
Commands Reference

Table of Contents

1. Git Commands 4

1. git config
2. git init
3. git clone
4. git status
5. git add
6. git commit
7. git log
8. git diff
9. git branch
10. git switch
11. git merge
12. git remote add
13. git push
14. git pull
15. git fetch
16. git stash
17. git rm
18. git mv
19. git restore
20. git restore --staged
21. git revert
22. git reset
23. git tag
24. git rebase
25. git cherry-pick
26. git reflog
27. git blame
28. git clean
29. git remote
30. git push --force

2. GitHub CLI (gh) Commands.....12

31. gh auth login
32. gh repo create
33. gh repo clone
34. gh repo view
35. gh repo list
36. gh repo fork
37. gh issue (list / create / view / close / reopen)

38. gh pr (create / list / view / merge / checkout / review)
39. gh workflow (list / run / view / enable / disable)
40. gh run (list / view / watch / cancel / rerun)
41. gh release (list / create / view / download)
42. gh secret (list / set / delete)
43. gh variable (list / set / delete)
44. gh search (repos / issues / prs / code)
45. gh api

3. Summary Table — All Commands.....16

1. Git Commands

Git is the version-control tool that runs on your computer. The commands below cover configuration, everyday workflow, branching, history, and advanced operations.

1. git config

Configures your Git username, email, and other global settings. This information is attached to every commit you make.

Syntax:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git config --list
```

Example:

```
git config --global user.name "Isha"
git config --global user.email "isha@example.com"
git config --list
```

Output:

```
user.name=Isha
user.email=isha@example.com
```

2. git init

Creates a new, empty Git repository in the current folder by adding a hidden .git directory.

Syntax:

```
git init
```

Example:

```
mkdir my-project
cd my-project
git init
```

Output:

```
Initialized empty Git repository in /path/my-project/.git/
```

3. git clone

Downloads (copies) an existing remote repository, along with its full history, to your computer.

Syntax:

```
git clone <repository-url>
```

Example:

```
git clone https://github.com/user/project.git
cd project
```

Output:

```
Cloning into 'project'...
remote: Enumerating objects...
Receiving objects: 100% done.
```

4. git status

Shows the current state of the working directory: which files are modified, staged, or untracked.

Syntax:

```
git status
```

Example:

```
git status
```

Output:

```
On branch main
Changes not staged for commit:
  modified:   index.html
Untracked files:
  style.css
```

5. git add

Moves changes from the working directory into the staging area, preparing them to be committed.

Syntax:

```
git add <file>
git add .
git add -A
```

Example:

```
git add index.html style.css
# or add everything
git add .
```

Output:

```
(no output on success; use git status to confirm files are staged)
```

6. git commit

Saves the currently staged changes to the repository's history as a new commit with a message.

Syntax:

```
git commit -m "message"
```

Example:

```
git commit -m "Add homepage"
```

Output:

```
[main a1b2c3d] Add homepage
 2 files changed, 40 insertions(+)
```

7. git log

Displays the commit history of the repository, from newest to oldest.

Syntax:

```
git log
git log --oneline
```

Example:

```
git log --oneline
```

Output:

```
a12bc34 Add homepage
b45de67 Fix navbar
c78fg90 Initial commit
```

8. git diff

Shows the line-by-line differences between the working directory and the last commit (or staged changes).

Syntax:

```
git diff
git diff --staged
```

Example:

```
git diff
```

Output:

```
-<h1>Old title</h1>
+<h1>New title</h1>
```

9. git branch

Lists, creates, or deletes branches in the repository.

Syntax:

```
git branch
git branch <name>
git branch -d <name>
```

Example:

```
git branch feature-login
git branch -d feature-login
```

Output:

```
* main
  feature-login
```

10. git switch

Switches your working directory to another existing branch, or creates a new one and switches to it.

Syntax:

```
git switch <branch>
git switch -c <new-branch>
```

Example:

```
git switch -c feature-login
```

Output:

```
Switched to a new branch 'feature-login'
```

11. git merge

Combines the history of one branch into the branch you are currently on.

Syntax:

```
git merge <branch>
```

Example:

```
git switch main
git merge feature-login
```

Output:

```
Updating a1b2c3..d4e5f6
Fast-forward
 index.html | 5 +++++
```

12. git remote add

Connects your local repository to a remote repository (e.g. on GitHub) under a chosen name, usually 'origin'.

Syntax:

```
git remote add origin <url>
git remote -v
```

Example:

```
git remote add origin https://github.com/user/my-project.git
git remote -v
```

Output:

```
origin  https://github.com/user/my-project.git (fetch)
origin  https://github.com/user/my-project.git (push)
```

13. git push

Uploads your local commits to the connected remote repository so others can see them.

Syntax:

```
git push
git push -u origin main
```

Example:

```
git push -u origin main
```

Output:

```
Enumerating objects: 5, done.
To https://github.com/user/my-project.git
 * [new branch]      main -> main
```

14. git pull

Downloads changes from the remote repository and merges them into your current local branch.

Syntax:

```
git pull
git pull origin main
```

Example:

```
git pull origin main
```

Output:

```
Updating a1b2c3..d4e5f6
Fast-forward
 style.css | 3 +++
```

15. git fetch

Downloads commits and information from the remote repository but does NOT merge them into your branch.

Syntax:

```
git fetch
git fetch origin
```

Example:

```
git fetch origin
```

Output:

```
remote: Enumerating objects...
   a1b2c3..d4e5f6  main -> origin/main
```

16. git stash

Temporarily saves changes that are not ready to be committed, so you can switch context and restore them later.

Syntax:

```
git stash
git stash list
git stash pop
```

Example:

```
git stash
git stash list
git stash pop
```

Output:

```
Saved working directory and index state WIP on main: a1b2c3 Add homepage
```

17. git rm

Removes a file from both the working directory and Git tracking.

Syntax:

```
git rm <file>
git rm -r <folder>
```

Example:

```
git rm test.txt
```

Output:

```
rm 'test.txt'
```

18. git mv

Renames or moves a tracked file, updating Git's records at the same time.

Syntax:

```
git mv <old> <new>
```

Example:

```
git mv old.txt new.txt
```

Output:

```
(no output on success; git status shows the rename)
```

19. git restore

Discards uncommitted changes made to a file, reverting it back to its last committed state.

Syntax:

```
git restore <file>
git restore .
```

Example:

```
git restore index.html
```

Output:

```
(no output; file content is reverted - use with caution)
```

20. git restore --staged

Removes a file from the staging area without deleting the actual changes in the working directory.

Syntax:

```
git restore --staged <file>
```

Example:

```
git add index.html
git restore --staged index.html
```

Output:

```
(file moves from 'staged' back to 'modified' in git status)
```

21. git revert

Creates a brand-new commit that undoes the changes made by an earlier commit, keeping history intact.

Syntax:

```
git revert <commit-id>
```

Example:

```
git revert a12bc34
```

Output:

```
[main 9f8e7d6] Revert "Add homepage"
 1 file changed, 2 deletions(-)
```

22. git reset

Moves the current branch pointer to a different commit. --soft keeps changes staged, --mixed unstages them, --hard discards them completely.

Syntax:

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
```

Example:

```
git reset --hard HEAD~1
```

Output:

```
HEAD is now at b45de67 Fix navbar
```

23. git tag

Creates a label (usually for a release version) on a specific commit.

Syntax:

```
git tag <name>
git push origin <name>
git push origin --tags
```

Example:

```
git tag v1.0
git push origin v1.0
```

Output:

```
To https://github.com/user/project.git
 * [new tag]         v1.0 -> v1.0
```

24. git rebase

Re-applies your commits on top of another branch, creating a cleaner, linear project history.

Syntax:

```
git rebase <branch>
git rebase -i HEAD~3
```

Example:

```
git switch feature-login
git rebase main
```

Output:

```
Successfully rebased and updated refs/heads/feature-login.
```

25. git cherry-pick

Applies the changes from one specific commit (from another branch) onto your current branch.

Syntax:

```
git cherry-pick <commit-id>
```

Example:

```
git cherry-pick a12bc34
```

Output:

```
[main 3c4d5e6] Add homepage
 Date: ...
 1 file changed, 10 insertions(+)
```

26. git reflog

Shows a log of every place HEAD has pointed to, including commits no longer visible in normal history. Useful for recovery.

Syntax:

```
git reflog
```

Example:

```
git reflog
```

Output:

```
a1b2c3d HEAD@{0}: commit: Add homepage
b45de67 HEAD@{1}: checkout: moving to main
```

27. git blame

Shows who last modified each line of a file, and in which commit.

Syntax:

```
git blame <file>
```

Example:

```
git blame app.py
```

Output:

```
a1b2c3d (Isha 2026-09-01) print("hello")
```

28. git clean

Permanently removes untracked files (and optionally directories) from the working directory.

Syntax:

```
git clean -f
git clean -fd
```

Example:

```
git clean -fd
```

Output:

```
Removing temp.log
Removing build/
```

29. git remote

Manages the set of remote repositories connected to your local project (view, add, remove, rename, change URL).

Syntax:

```
git remote -v
git remote remove origin
git remote set-url origin <url>
```

Example:

```
git remote set-url origin https://github.com/user/new-project.git
```

Output:

```
(no output on success; git remote -v shows the updated URL)
```

30. git push --force

Forces your local branch history onto the remote, overwriting the remote history. Dangerous - can erase others' commits.

Syntax:

```
git push --force
git push --force-with-lease
```

Example:

```
git push --force-with-lease
```

Output:

```
+ a1b2c3...d4e5f6 main -> main (forced update)
```

2. GitHub CLI (gh) Commands

GitHub is the online platform where Git repositories are hosted and shared. The GitHub CLI (gh) must be installed separately from Git, and lets you manage repositories, issues, pull requests, releases, and Actions directly from the terminal.

31. gh auth login

Logs your terminal into your GitHub account so other gh commands can authenticate.

Syntax:

```
gh auth login
gh auth status
```

Example:

```
gh auth login
```

Output:

```
✓ Logged in to github.com as your-username
```

32. gh repo create

Creates a new repository directly on GitHub from the command line.

Syntax:

```
gh repo create <name> [--public|--private]
```

Example:

```
gh repo create my-project --public
```

Output:

```
✓ Created repository user/my-project on GitHub
```

33. gh repo clone

Clones a GitHub repository to your computer using the GitHub CLI.

Syntax:

```
gh repo clone <owner>/<repo>
```

Example:

```
gh repo clone username/my-project
```

Output:

```
Cloning into 'my-project'...
```

34. gh repo view

Displays details about a GitHub repository, such as its description and README.

Syntax:

```
gh repo view [owner/repo]
```

Example:

```
gh repo view
```

Output:

```
user/my-project
A sample project
...
```

35. gh repo list

Lists repositories belonging to a user or organization.

Syntax:

```
gh repo list [owner]
```

Example:

```
gh repo list
```

Output:

```
user/my-project    Public
user/old-project   Private
```

36. gh repo fork

Creates your own copy (fork) of someone else's repository under your account.

Syntax:

```
gh repo fork <owner>/<repo>
```

Example:

```
gh repo fork username/project
```

Output:

```
✓ Created fork you/project
```

37. gh issue (list / create / view / close / reopen)

Manages GitHub Issues directly from the terminal.

Syntax:

```
gh issue list
gh issue create
gh issue view <number>
gh issue close <number>
gh issue reopen <number>
```

Example:

```
gh issue create
gh issue view 1
gh issue close 1
```

Output:

```
#1  Fix login bug   open
```

38. gh pr (create / list / view / merge / checkout / review)

Manages GitHub Pull Requests: creating, listing, viewing, merging, checking out, and reviewing them.

Syntax:

```
gh pr create
gh pr list
gh pr view <number>
gh pr merge <number>
gh pr checkout <number>
gh pr review <number>
```

Example:

```
gh pr create
gh pr merge 1
```

Output:

```
✓ Pull request #1 created
✓ Merged pull request #1
```

39. gh workflow (list / run / view / enable / disable)

Manages GitHub Actions workflows: listing, running manually, viewing, enabling, or disabling them.

Syntax:

```
gh workflow list
gh workflow run <file>
gh workflow view <file>
```

Example:

```
gh workflow run deploy.yml
```

Output:

```
✓ Created workflow_dispatch event for deploy.yml
```

40. gh run (list / view / watch / cancel / rerun)

Manages individual GitHub Actions workflow runs.

Syntax:

```
gh run list
gh run view <id>
gh run watch
gh run cancel <id>
gh run rerun <id>
```

Example:

```
gh run list
```

Output:

```
✓ deploy   main   push   2m ago
```

41. gh release (list / create / view / download)

Manages GitHub releases - packaged, versioned snapshots of your project.

Syntax:

```
gh release list
gh release create <tag>
gh release view <tag>
gh release download <tag>
```

Example:

```
gh release create v1.0
```

Output:

```
✓ Created release v1.0 for user/my-project
```

42. gh secret (list / set / delete)

Manages encrypted secrets used by GitHub Actions workflows (e.g. API keys, credentials).

Syntax:

```
gh secret list
gh secret set <NAME>
gh secret delete <NAME>
```

Example:

```
gh secret set AWS_ACCESS_KEY_ID
```

Output:

```
? Paste your secret
✓ Set secret AWS_ACCESS_KEY_ID
```

43. gh variable (list / set / delete)

Manages non-secret configuration variables used by GitHub Actions workflows.

Syntax:

```
gh variable list
gh variable set <NAME> --body "value"
gh variable delete <NAME>
```

Example:

```
gh variable set ENVIRONMENT --body "production"
```

Output:

```
✓ Set variable ENVIRONMENT for user/my-project
```

44. gh search (repos / issues / prs / code)

Searches GitHub directly from the terminal for repositories, issues, pull requests, or code.

Syntax:

```
gh search repos <query>
gh search issues <query>
gh search prs <query>
gh search code <query>
```

Example:

```
gh search repos docker
```

Output:

```
moby/moby       Moby Project - a collaborative project...
docker/compose  Define and run multi-container apps
```

45. gh api

Sends authenticated requests directly to the GitHub REST API from the terminal.

Syntax:

```
gh api <endpoint>
```

Example:

```
gh api user
gh api repos/username/project
```

Output:

```
{
  "login": "username",
  "id": 123456
}
```

3. Summary Table — All Commands

A quick-reference table of every command covered in this document.

Command	Description
`git config`	Set Git username, email, and preferences
`git init`	Create a new local Git repository
`git clone`	Download an existing repository
`git status`	Show changed / staged / untracked files
`git add`	Stage changes for the next commit
`git commit`	Save staged changes to history
`git log`	View commit history
`git diff`	Show unstaged or staged changes
`git branch`	List, create, or delete branches
`git switch`	Change to another branch
`git merge`	Combine one branch into another
`git remote add`	Connect local repo to a remote (GitHub)
`git push`	Upload local commits to the remote
`git pull`	Download and merge remote changes
`git fetch`	Download remote changes without merging
`git stash`	Temporarily save uncommitted changes
`git rm`	Remove a file from Git and disk
`git mv`	Rename or move a tracked file
`git restore`	Discard uncommitted changes
`git restore --staged`	Unstage a file
`git revert`	Undo a commit by creating a new commit
`git reset`	Move branch pointer / undo commits
`git tag`	Mark a specific commit as a version
`git rebase`	Reapply commits on top of another branch
`git cherry-pick`	Apply a specific commit to current branch
`git reflog`	View history of HEAD movements
`git blame`	Show who last edited each line
`git clean`	Remove untracked files/directories
`git remote`	Manage connected remote repositories
`git push --force`	Force local history onto the remote
`gh auth login`	Log the CLI into GitHub
`gh repo create`	Create a new GitHub repository
`gh repo clone`	Clone a GitHub repository
`gh repo view`	View repository details
`gh repo list`	List repositories
`gh repo fork`	Fork another user's repository
`gh issue ...`	Create/list/view/close GitHub issues
`gh pr ...`	Create/list/view/merge pull requests
`gh workflow ...`	Manage GitHub Actions workflows
`gh run ...`	View / manage Actions workflow runs
`gh release ...`	Manage GitHub releases
`gh secret ...`	Manage GitHub Actions secrets
`gh variable ...`	Manage GitHub Actions variables
`gh search ...`	Search repos, issues, PRs, code
`gh api`	Call the GitHub REST API directly